

CS-405: Deep Learning

Week 3 - Introduction to Convolutional Neural Networks

Dr. Sajjad Hussain

February 3rd, 2026

Why Convolutional Neural Networks?

- **Problem with Fully Connected Networks for Images:**
 - **Parameter explosion:** For a 1000×1000 image with 1M pixels
 - First hidden layer with 1M neurons $\rightarrow$ 1 trillion parameters!
 - **Spatial structure loss:** Flattening destroys local pixel relationships
 - **No translation invariance:** A shifted object requires completely new learning
 - **Computationally infeasible** for real-world image processing
- **Convolutional Neural Networks solve these problems by:**
 - **Local connectivity:** Neurons connect only to small regions of input
 - **Parameter sharing:** Same weights used across all spatial positions
 - **Hierarchical features:** Learn from simple to complex patterns

- **Hubel & Wiesel's Experiments (1959):**

- Studied cat visual cortex
- Discovered hierarchical organization
- Different neurons respond to different visual patterns

- **Key Findings:**

- **Simple cells:** Respond to edges at specific orientations
- **Complex cells:** Respond to edges regardless of exact position
- **Hierarchical processing:** From edges → shapes → objects
- **Receptive fields:** Each neuron responds to specific region of visual field

- **CNN Analogy:**

- Early conv layers = Simple cells (edge detectors)
- Pooling layers = Complex cells (position invariance)
- Deep layers = Object recognition

Convolution: Mathematical Definition

Discrete 2D Convolution:

$$S[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k X[i + u, j + v] \cdot K[u, v]$$

Where:

- X : Input image or feature map (size $N \times N$)
- K : Kernel or Filter (size $(2k + 1) \times (2k + 1)$)
 - Typically 3×3 , 5×5 , or 7×7
- S : Output feature map
- Operation: Element-wise multiplication followed by summation

Key Insight: The filter "slides" across the input, computing dot products at each position.

Convolution Operation

Input Image (5×5)

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

-1	0	1
-1	0	1
-1	0	1

Filter (3×3)

6	6	6
6	6	6
6	6	6

Output (3×3)

Step 1: Place the 3×3 filter on the top-left 3×3 region of the input image.

Step 2: Multiply element-wise filter value with the corresponding input pixel and add the values to obtain a single scalar output.

Step 3: Store this value in the output at the corresponding location.

Step 4: Move the filter one pixel to the right and repeat Steps 2–3 until the filter reaches the right edge.

Step 5: Move the filter one pixel down, return to the left, and repeat until the entire input image is covered.

Multiple Filters: Learning Different Features

- **Key Concept:** Each filter learns to detect a different type of feature
- **Example Filters Learned by CNNs:**

- **Vertical edge detectors:** $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$
- **Horizontal edge detectors:** $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$
- **Diagonal edge detectors:** Various orientations
- **Blob detectors:** Center-surround patterns
- **Color detectors:** Specific color combinations
- **Texture detectors:** Repeated patterns

- **Feature Maps:**

- Each filter produces one feature map
- Bright regions indicate strong presence of that feature
- Dark regions indicate absence of that feature
- Stack of feature maps forms the output volume depth

Stride: Controlling How Filters Move

- **Definition:** Step size with which the filter moves across the input
- **Common Stride Values:**
 - **Stride = 1:** Filter moves 1 pixel at a time
 - Most detailed output
 - Maximum computation
 - Output size similar to input
 - **Stride = 2:** Filter moves 2 pixels at a time
 - Output approximately half the input size
 - Reduces computation by 75%
 - Common in practice
 - **Stride > 2:** Used for aggressive downsampling
- **Effects of Larger Stride:**
 - Reduces spatial dimensions of output
 - Decreases computational cost
 - May lose fine-grained information
 - Can cause aliasing effects
- **Trade-off:**
 - Smaller stride: Better accuracy, more computation
 - Larger stride: Faster processing, potential information loss

Padding: Preserving Spatial Information

- **Problem:** Convolution reduces spatial dimensions
 - Input: $N \times N$, Filter: $F \times F$, Output: $(N - F + 1) \times (N - F + 1)$
 - Information at borders used less frequently
 - Multiple layers cause significant shrinking
- **Solution:** Add pixels around the input border
- **Types of Padding:**
 - **Zero Padding:** Add zeros around border (most common)
 - **Reflection Padding:** Mirror image at borders
 - **Replication Padding:** Repeat edge pixels
- **Common Padding Strategies:**
 - **Valid Padding (No Padding):** $P = 0$
 - Output shrinks: $O = N - F + 1$
 - **Same Padding:** Choose P so output size equals input size
 - When stride=1: $P = \frac{F-1}{2}$
 - Output: $O = N$ (preserves spatial dimensions)

Complete Output Dimension Formula

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Where:

- O = Output size (width or height)
- N = Input size (width or height)
- F = Filter size (width or height)
- P = Padding amount
- S = Stride

Important Notes:

- Formula applies separately to width and height

Convolution with Multi-Channel Inputs (RGB)

- Real images usually have multiple channels (e.g., RGB has 3 channels).
- Each channel represents a different feature dimension:
 - Red intensity
 - Green intensity
 - Blue intensity
- A convolutional filter must span **all input channels**.
- For an RGB image, a filter has size:

$$k \times k \times 3$$

Key idea: Spatial size slides, channel depth matches input.

RGB Filter Structure

- An RGB filter consists of **three separate 2D filters**:

$$W = \begin{bmatrix} W^{(R)} \\ W^{(G)} \\ W^{(B)} \end{bmatrix}$$

- Each filter operates on its corresponding input channel.
- All channel-wise results are summed to produce one output value.

$$\text{Output value} = \sum_{c \in \{R, G, B\}} \sum_{i, j} X_{i, j}^{(c)} \cdot W_{i, j}^{(c)}$$

One filter $\Rightarrow$ one feature map

Steps: Convolution for RGB Images

- 1 Select a $k \times k \times 3$ patch from the input image.
- 2 Perform element-wise multiplication for each channel:
 - Red channel $\rightarrow$ Red filter
 - Green channel $\rightarrow$ Green filter
 - Blue channel $\rightarrow$ Blue filter
- 3 Sum all values across **space and channels**.
- 4 Add a scalar bias term.
- 5 Apply an activation function.

Result: One scalar value in the output feature map.

CNN Convolution as a Neural Network Operation

- Convolution is a special case of a linear transformation.
- Flatten the local RGB patch into a vector X .
- Flatten the filter weights into a vector W .

$$z = WX + b$$

$$a = g(z)$$

- z = pre-activation (convolution result + bias)
- $g(\cdot)$ = activation function (e.g., ReLU)
- a = activated feature map value

CNNs = local, shared-weight neural networks

Convolution Over Volume (Multi-Channel Input)

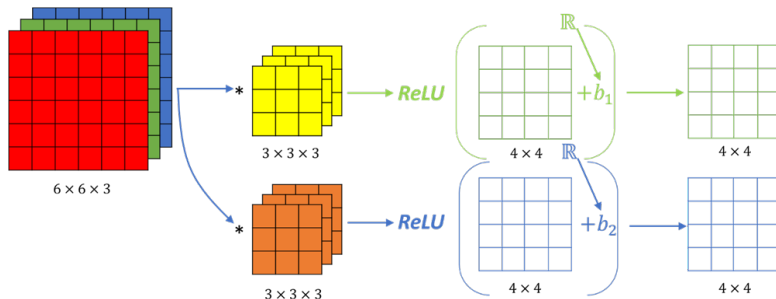


Figure adapted from Andrew Ng, *Convolutional Neural Networks*, DeepLearning.AI.

Single Convolution Layer: Dimensions and Parameters

Input to layer ℓ :

$$\text{Input: } n_H^{[\ell-1]} \times n_W^{[\ell-1]} \times n_C^{[\ell-1]}$$

Hyperparameters of layer ℓ :

- Filter (kernel) size: $f^{[\ell]} \times f^{[\ell]}$
- Stride: $s^{[\ell]}$
- Padding: $p^{[\ell]}$
- Number of filters: $n_C^{[\ell]}$

Single Convolution Layer: Dimensions and Parameters

Filter Weights:

$$W^{[\ell]} = f^{[\ell]} \times f^{[\ell]} \times n_C^{[\ell-1]} \times n_C^{[\ell]}$$

Bias:

$$b^{[\ell]} = 1 \times 1 \times 1 \times n_C^{[\ell]}$$

Output (feature map) dimensions:

$$n_H^{[\ell]} = \left\lfloor \frac{n_H^{[\ell-1]} - f^{[\ell]} + 2p^{[\ell]}}{s^{[\ell]}} \right\rfloor + 1$$

$$n_W^{[\ell]} = \left\lfloor \frac{n_W^{[\ell-1]} - f^{[\ell]} + 2p^{[\ell]}}{s^{[\ell]}} \right\rfloor + 1$$

$$\text{Output: } n_H^{[\ell]} \times n_W^{[\ell]} \times n_C^{[\ell]}$$

Activations:

$$Z^{[\ell]} = W^{[\ell]} * A^{[\ell-1]} + b^{[\ell]}, \quad A^{[\ell]} = g(Z^{[\ell]})$$

Pooling Operations: Achieving Invariance

- **The Need for Pooling:**
 - Convolution preserves exact spatial locations (equivariance)
 - But for classification, we often want invariance to small translations (pooling ignores small shifts)
 - Need to reduce spatial dimensions to control parameters
- **Pooling Benefits:**
 - ① **Translation Invariance:** Small shifts don't affect output
 - ② **Dimensionality Reduction:** Downsample feature maps
 - ③ **Parameter Reduction:** Less computation in later layers
 - ④ **Overfitting Prevention:** Reduce spatial resolution
 - ⑤ **Robustness:** To small distortions and noise
- **Common Pooling Types:**
 - **Max Pooling:** Take maximum value in window
 - **Average Pooling:** Take average value in window
 - **Global Average Pooling:** Average entire feature map
 - **Strided Convolution:** Learnable alternative to pooling
- **Typical Parameters:**
 - Pool size: Usually 2×2 or 3×3
 - Stride: Usually equals pool size (non-overlapping)

Max Pooling Operation

Input Feature Map (4×4)

2	5	1	3
4	9	2	7
3	6	8	1
1	4	5	2



9	7
6	8

Max Pool Output (2×2)

Calculations:

Top-left: $\max(2, 5, 4, 9) = 9$

Top-right: $\max(1, 3, 2, 7) = 7$

Bottom-left: $\max(3, 6, 1, 4) = 6$

Bottom-right: $\max(8, 1, 5, 2) = 8$

Average Pooling Operation

Input Feature Map (4×4)

2	5	1	3
4	9	2	7
3	6	8	1
1	4	5	2



5.0	3.25
3.5	4.0

Avg Pool Output (2×2)

Calculations:

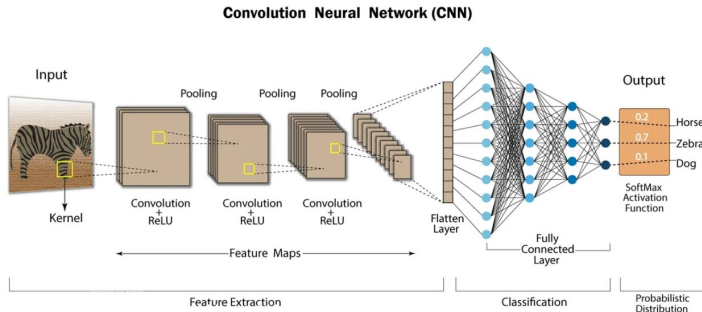
Top-left: $(2+5+4+9)/4 = 5.0$

Top-right: $(1+3+2+7)/4 = 3.25$

Bottom-left: $(3+6+1+4)/4 = 3.5$

Bottom-right: $(8+1+5+2)/4 = 4.0$

Architecture of a Simple Convolutional Neural Network



- 1 Input Image
- 2 Convolution + Activation
- 3 Pooling
- 4 Repeat Conv-Pool blocks (increasing depth)
- 5 Flattening
- 6 Fully Connected Layers
- 7 Output Layer (Sigmoid / Softmax)

Figure source: Nafiz Shahriar, *What is a Convolutional Neural Network (CNN)?*, [LinkedIn Pulse](#).

What Happens at Each Stage of a CNN

- **Input Layer:**
 - Raw pixel values (e.g., $H \times W \times 3$ for RGB images)
- **Convolution + Activation:**
 - Learns local features (edges, textures, patterns)
 - Number of filters controls feature diversity (depth)
- **Pooling Layer:**
 - Reduces spatial dimensions
 - Reduces computation and overfitting
- **Deeper Conv–Pool Blocks:**
 - Combine simple features into complex patterns (Edges $\rightarrow$ Shapes $\rightarrow$ Object Parts $\rightarrow$ Objects)
 - Spatial size decreases, depth increases
- **Flattening:**
 - Converts 3D feature maps into a 1D feature vector
- **Fully Connected Layers:**
 - Perform high-level reasoning
 - Combine learned features for decision making
- **Output Layer:**
 - Sigmoid for binary and softmax for multi-class classification

LeNet-5: The First Successful CNN (1998)

- **Historical Context:**

- Developed by Yann LeCun et al. in 1998
- Designed for handwritten digit recognition (MNIST)
- Used in banking for check reading
- Pioneered the CNN architecture pattern

- **Key Contributions:**

- First practical application of backpropagation to CNNs
- Established Convolution $\Rightarrow$ Pooling $\Rightarrow$ FC pattern
- Demonstrated effectiveness of hierarchical feature learning
- Achieved 0.8% error rate on MNIST (state-of-the-art at time)

- **Technical Details:**

- Input: 32×32 grayscale images
- 2 convolutional layers, 2 pooling layers, 2 fully connected layers
- Total parameters: 60,000
- Activation functions: Sigmoid/tanh
- Training: Stochastic gradient descent

- **Impact:**

- Proved CNNs could solve real-world problems
- Set foundation for future CNN architectures

LeNet-5 Architecture

